Implementing Network Security

in a Virtual Environment

A Defense-in-Depth Lab Guide

Segmentation • IDS • VPN • DNS filtering • monitoring • centralized logging

1. Introduction

Network security is rarely a single control. A modern network is defended by a stack of overlapping mechanisms — segmentation that limits where an attacker can move, firewalls that decide what crosses each boundary, intrusion detection that surfaces what looks wrong, encryption that protects traffic in transit, DNS filtering that blocks resolution of known-bad domains, monitoring that shows what is actually happening, and centralized logging that ties it all together. Any one of those controls can fail; the others compensate. That layered approach is what "defense in depth" means in practice.

This lab builds a small but realistic environment where each of those layers is configured, exercised, and validated by hand. The goal is not to memorize the click paths in any particular product — it is to understand which problem each control solves, how it fails, and how the controls combine to produce a result that no individual mechanism could produce alone.

Learning Objectives

After completing this lab, you will be able to:

- Design and operate a multi-zone virtualized network with controlled paths between zones.
- Configure and tune a network intrusion detection system, interpret its alerts, and tell a true positive from a noisy rule.
- Stand up an encrypted VPN for remote access and verify that traffic is actually protected end to end.
- Block resolution of malicious and unwanted domains at the DNS layer with Pi-hole.
- Capture, filter, and analyze packets to investigate suspicious behavior.
- Forward logs from network devices to a central collector and write basic queries against them.
- Walk an end-to-end attack scenario through every layer of the stack and explain which control caught what.

How This Lab Builds on Earlier Work

If you have completed the companion access-control lab using pfSense, you can move quickly through Part 1 and reuse the existing virtual machines. If this is your first network lab, Part 1 will walk you

through the foundation from scratch. Either way, the work in this lab extends the environment with new tools, new VMs, and new controls layered on top of the firewall rules already in place.

2. Prerequisites

Hardware

Resource	Recommended	Minimum
Host CPU	8 cores with VT-x or AMD-V	4 cores with virtualization extensions
Host RAM	32 GB	16 GB
Host Storage	150 GB free SSD	80 GB free disk
Hypervisor	VirtualBox 7.x (free) or VMware Workstation Player	Either of the above
Host networks	Four host-only networks for zone separation	Three host-only networks

Table 1 – Hardware Prerequisites

Software to Download

1. VirtualBox 7.x — https://www.virtualbox.org/wiki/Downloads
2. pfSense Community Edition installer ISO — https://www.pfsense.org/download/
3. Ubuntu Server 22.04 LTS ISO — https://ubuntu.com/download/server
4. Ubuntu Desktop 22.04 LTS ISO — https://ubuntu.com/download/desktop
5. Kali Linux installer ISO (or Ubuntu with security tools added) — https://www.kali.org/get-kali/ (used in the validation scenario as a benign attacker)
6. Wireshark — https://www.wireshark.org/download.html (installed on the LAN client during Part 6)

Background Knowledge

- IPv4 subnetting: you should be comfortable reading 192.168.10.0/24 and identifying which addresses fall inside it.
- Basic Linux command line: navigating directories, editing files, running services.
- How a packet typically traverses a stateful firewall (interface, action, state table).

- The general purpose of TCP, UDP, ICMP, DNS, HTTP, and HTTPS.

> **BEFORE YOU START**
> Take a snapshot of every VM after each major step. Many of the configurations in this lab can be reverted faster than they can be repaired, and the discipline of snapshotting between layers makes the lab much more forgiving when you experiment.

3. Lab Topology

The lab models a small organization with four security zones, each separated by the pfSense firewall. Two zones host endpoints, one is dedicated to public-facing services, and one hosts the security tooling used to monitor and protect the rest of the environment.

Network Zones

Zone	Subnet	pfSense Interface	Purpose
WAN	DHCP from host	WAN (em0)	Path to the internet via the host's NAT
LAN	192.168.10.0/24	LAN (em1)	Trusted internal users
DMZ	192.168.20.0/24	OPT1 (em2)	Public-facing services (web, etc.)
MGMT	192.168.40.0/24	OPT2 (em3)	Security tooling (Pi-hole, log collector)

Table 2 – Network Zones

Virtual Machines

VM Name	Operating System	Role
pfSense-FW	pfSense CE 2.7+	Router, firewall, IDS host (Suricata), VPN server
LAN-Client	Ubuntu Desktop 22.04	Trusted user workstation (192.168.10.10)
DMZ-Web	Ubuntu Server 22.04	Apache web server in DMZ (192.168.20.10)
MGMT-Tools	Ubuntu Server 22.04	Pi-hole DNS sinkhole and rsyslog collector (192.168.40.10)
Attacker	Kali Linux (or Ubuntu)	Used in Part 8 to validate detection (placed on LAN for now)

Table 3 – Virtual Machine Lists

Defense-in-Depth Mapping

Each part of the lab corresponds to a specific layer in the defense-in-depth model. Keeping this mapping in mind helps explain why a given control is being added at a given step.

Layer	Control	What It Protects Against
Network architecture	Zone segmentation	Lateral movement after a host is compromised
Perimeter / boundary	Firewall rules	Unauthorized cross-zone traffic
Detection	Suricata IDS	Known attack patterns and policy violations
Confidentiality	WireGuard VPN	Eavesdropping on remote-access traffic
Resolution	Pi-hole DNS filtering	Resolution of known-bad domains
Visibility	ntopng + Wireshark	Blind spots; unknown unknowns
Forensics	Centralized logging	Loss of evidence after a compromise

Table 4 – Defense-in-Depth Mapping

PART 1

Build the Foundation

Estimated time: **60 minutes**

This part stands up the firewall and the four host-only networks. If you have completed the prior access-control lab, you can reuse the existing pfSense-FW VM and skip directly to Step 1.4 to add the new MGMT interface.

Step 1.1 Create four host-only networks

7. Open VirtualBox → File → Tools → Network Manager → Host-only Networks.
8. Create four networks with these adapter settings, and disable DHCP on all of them (pfSense will run DHCP):

Network Name	IPv4 Address	Netmask	DHCP
LAB-LAN	192.168.10.254	255.255.255.0	Disabled
LAB-DMZ	192.168.20.254	255.255.255.0	Disabled

Network Name	IPv4 Address	Netmask	DHCP
LAB-MGMT	192.168.40.254	255.255.255.0	Disabled
LAB-ATTACK	192.168.50.254	255.255.255.0	Disabled

Table 5 – Network Interface Zones

Step 1.2 Build the pfSense VM

9. Create a VM named pfSense-FW: Type BSD, Version FreeBSD (64-bit), 1024 MB RAM, 16 GB disk.
10. Add four network adapters before booting:
    - Adapter 1: NAT (WAN, internet via the host)
    - Adapter 2: Host-only Adapter (LAB-LAN)
    - Adapter 3: Host-only Adapter (LAB-DMZ)
    - Adapter 4: Host-only Adapter (LAB-MGMT)
11. Mount the pfSense ISO and install with default options (Auto ZFS layout).

Step 1.3 Assign and configure interfaces

After the first boot, the pfSense console will prompt for interface assignment. Decline VLAN setup, then assign:

- WAN: em0 (NAT)
- LAN: em1 (LAB-LAN, IP 192.168.10.1/24, DHCP 192.168.10.100–200)
- OPT1: em2 (LAB-DMZ, IP 192.168.20.1/24, DHCP 192.168.20.100–200)
- OPT2: em3 (LAB-MGMT, IP 192.168.40.1/24, DHCP 192.168.40.100–200)

Step 1.4 Bring up the OPT1 and OPT2 interfaces in the GUI

By default, pfSense leaves OPT interfaces disabled. From the LAN client, log in to https://192.168.10.1 and:

12. Navigate to Interfaces → Assignments. Confirm the four interfaces (WAN, LAN, OPT1, OPT2) are present.
13. For each OPT interface, click the interface, check Enable, and rename the interface for clarity (DMZ for OPT1, MGMT for OPT2). Save and Apply Changes.
14. Configure each interface with a static IPv4 address and the DHCP range you specified in the console.

Step 1.5 Build the endpoint VMs

Create the four endpoint VMs and attach each to the appropriate host-only network. Install Ubuntu (Desktop or Server as indicated) using default options. After install, confirm each VM received a DHCP address from pfSense by running:

```
ip a
ip route
ping -c 3 <pfSense IP for that interface>
ping -c 3 8.8.8.8
```

Set static IPs on the server VMs (DMZ-Web, MGMT-Tools) so the rules and tools you configure later are predictable. The desktop VMs (LAN-Client, Attacker) can stay on DHCP.

> **MINIMUM FIREWALL POSTURE**
>
> Before continuing, write a baseline rule set on every interface: allow LAN to reach the internet, allow LAN to reach MGMT (so you can administer the security tools), block all other inter-zone traffic. The rest of this lab assumes a deny-by-default posture. If you completed the access-control lab, your existing LAN and DMZ rules are a good starting point.

PART 2

Segmentation and Inter-Zone Policy

Estimated time: **45 minutes**

Segmentation is the architectural foundation of network security. A flat network gives an attacker who compromises one host the ability to reach every other host on the same broadcast domain. A segmented network makes each cross-zone hop a separate decision point, which is what gives the rest of the controls in this lab something to enforce.

Inter-Zone Access Policy

Author the following policy on pfSense. Each row is a rule on the source interface.

From	To	Action	Service	Notes
LAN	Internet	**Pass**	DNS, HTTP/S, ICMP	Standard user activity
LAN	DMZ	**Pass**	HTTP/S to DMZ-Web	Reach the web server
LAN	MGMT	**Pass**	HTTPS, SSH	Admin access to security tools
LAN	MGMT	**Pass**	DNS to Pi-hole	DNS resolution
DMZ	Internet	**Pass**	HTTP/S, DNS	Outbound updates only
DMZ	LAN	**Block**	Any	Prevent pivot from DMZ
DMZ	MGMT	**Block**	Any	Prevent pivot from DMZ
MGMT	Any	**Pass**	HTTP/S, DNS, syslog	Tools need to function
MGMT	LAN/DMZ	**Block**	Any	Tools should not initiate

Table 6 – Inter-Zone ACL Policy

Implementation Notes

- **Use aliases.** Create aliases for LAN_Net, DMZ_Net, MGMT_Net, DMZ_Web_Server, Pi_Hole, and Web_Ports. Rules referencing aliases are easier to read and easier to update.
- **Order matters.** Rules are evaluated top-down on each interface; the first match wins. Block rules that protect critical zones go above pass rules.

- **Log denies.** Add an explicit final block rule on each interface with logging enabled. Without the explicit log, denied packets disappear silently.

Validation

From each VM, attempt the listed cross-zone connection and confirm the expected behavior. Use ping, curl, or nc.

```
# From LAN-Client
curl -m 5 http://192.168.20.10      # Pass
curl -m 5 http://192.168.40.10      # Pass (admin to MGMT)

# From DMZ-Web
ping -c 3 192.168.10.10             # Block (no reply)
ping -c 3 192.168.40.10             # Block (no reply)
sudo apt update                     # Pass (HTTP/S to internet)

# From MGMT-Tools
ping -c 3 192.168.10.10             # Block
```

PART 3

Intrusion Detection with Suricata

Estimated time: **75 minutes**

A firewall decides what is allowed; an IDS decides what is suspicious. Together they cover two different failure modes: a firewall protects you from packets that should not be there, while an IDS protects you from packets that look legitimate but are not. Suricata is a high-performance, multi-threaded IDS/IPS that ships as a pfSense package and reads the same Emerging Threats rule sets used in production by many organizations.

Step 3.1 Install the Suricata package

15. In the pfSense web GUI, navigate to System → Package Manager → Available Packages.
16. Find suricata, click Install, and confirm. The install takes 3–5 minutes.
17. After install, navigate to Services → Suricata.

Step 3.2 Configure global settings and rule sources

18. Click the Global Settings tab.

19. Enable ETOpen Emerging Threats Rules. This is the free community rule set; it is updated regularly and covers thousands of known attack patterns.
20. Optionally enable Snort GPLv2 Community Rules for additional coverage.
21. Set Update Interval to 1 day. Save.
22. Click the Updates tab and click Update. Wait for confirmation that rules downloaded successfully.

Step 3.3 Add the WAN interface

23. Click the Interfaces tab, then Add.
24. Select WAN as the interface.
25. Enable the interface.
26. Set Send Alerts to System Log to checked.
27. Leave the rest at defaults (Suricata on the WAN catches inbound attacks before they hit internal hosts) and Save.

Step 3.4 Select rules

Suricata installs thousands of rules by default. Most labs do not need all of them, and turning everything on produces a great deal of noise. For this lab, enable a focused set:

28. Click the WAN interface entry, then the WAN Categories tab.
29. Enable the following ET Open categories:
    - emerging-attack_response.rules
    - emerging-current_events.rules
    - emerging-exploit.rules
    - emerging-malware.rules
    - emerging-scan.rules
    - emerging-trojan.rules
    - emerging-web_server.rules
30. Save and Apply.

Step 3.5 Start Suricata

31. Return to the Interfaces tab. Find the WAN row and click the play icon to start Suricata on that interface.
32. Wait 30–60 seconds for Suricata to load rules. Confirm the status indicator turns green.

Step 3.6 Generate test traffic and confirm detection

From the LAN-Client, generate a request that ET rules are known to detect:

```
# This URL triggers a known ET signature designed for testing
curl http://testmyids.com/

# Expected response: "uid=0(root) gid=0(root) groups=0(root)"
# (this is a benign test string, not an actual exploit)
```

Then check Services → Suricata → Alerts. You should see one or more alerts referencing this connection. Click the magnifying glass icon next to an alert to see the full rule that matched, the source and destination IPs, and a packet excerpt.

> **FROM IDS TO IPS**
>
> Suricata becomes an Intrusion Prevention System (IPS) when you check Block Offenders on the interface settings. In a production environment that change must be made carefully — false positives now cause outages, not just noisy alerts. For this lab, leave Suricata in IDS mode (alert only) for the rest of the exercise. Switch to IPS mode only after you have tuned the rule set against several days of normal traffic.

Step 3.7 Tune one false positive

After running normal traffic for a while, examine the alert list and identify one alert that is clearly a false positive — for example, a routine HTTPS connection that matches a noisy heuristic rule. Disable that specific signature:

33. In the alert detail, note the SID (signature ID) of the rule.
34. Click Suppress this alert and choose By IP and SID to suppress only this combination.
35. Confirm the alert no longer fires for that source on the next test run.

Tuning is the work that turns an IDS from a noise generator into a useful tool. Production teams spend a meaningful share of their time on this loop.

PART 4

Encrypted Remote Access with WireGuard

Estimated time: **60 minutes**

WireGuard is a modern VPN protocol designed to be fast, simple to configure, and built on cryptographic primitives (Curve25519, ChaCha20-Poly1305, BLAKE2s) that are widely audited and well understood. pfSense ships a WireGuard package that lets you stand up a remote-access VPN in less than fifteen minutes.

Step 4.1 Install the WireGuard package

36. System → Package Manager → Available Packages → search WireGuard → Install.
37. After install, navigate to VPN → WireGuard.

Step 4.2 Create the server tunnel

38. Click the Tunnels tab → Add Tunnel.
39. Description: Lab-VPN.
40. Listen Port: 51820 (the default).
41. Click Generate to create the server's public/private key pair.
42. Interface Addresses: 10.99.99.1/24. This is the tunnel network — VPN clients will receive addresses in this range.
43. Save.

Step 4.3 Create a peer (client)

44. Click the Peers tab → Add Peer.
45. Description: laptop-001.
46. Tunnel: select the Lab-VPN tunnel created above.
47. Click Generate for the peer key pair, or paste a public key generated externally on the client device.
48. Allowed IPs: 10.99.99.2/32 (the address this peer will receive).
49. Save.

Step 4.4 Assign the WireGuard interface in pfSense

50. Interfaces → Assignments → Available network ports: select the WireGuard tunnel and click Add.
51. Click the new interface (likely OPT3), enable it, and save.
52. Apply changes.

Step 4.5 Open the firewall on WAN and tunnel interface

53. Firewall → Rules → WAN. Add a rule:
    - Action: Pass
    - Protocol: UDP
    - Source: any
    - Destination: WAN address, port 51820

- Description: Allow inbound WireGuard

54. Firewall → Rules → WG_Lab-VPN (the new tunnel interface). Add a rule:
    - Action: Pass
    - Protocol: any
    - Source: 10.99.99.0/24
    - Destination: any
    - Description: Allow VPN clients into the lab

In production you would constrain VPN clients to specific destinations rather than "any." For the lab, the broad rule is fine — least privilege for VPN endpoints is its own exercise.

Step 4.6 Configure a client and test

Install WireGuard on a client device (the host workstation, or a fourth VM). The configuration file looks like this — adjust addresses and keys to match your tunnel and peer:

```
[Interface]
PrivateKey = <client_private_key>
Address = 10.99.99.2/32
DNS = 192.168.40.10

[Peer]
PublicKey = <pfSense_server_public_key>
Endpoint = <pfSense_WAN_IP>:51820
AllowedIPs = 192.168.10.0/24, 192.168.20.0/24, 192.168.40.0/24
PersistentKeepalive = 25
```

55. Save the file as lab.conf and import it into the WireGuard client application.
56. Activate the tunnel.
57. Verify connectivity:

```
# From the client, after the tunnel is up
ping 10.99.99.1              # Should reach pfSense's tunnel IP
ping 192.168.10.1            # Should reach pfSense's LAN IP
curl -m 5 http://192.168.20.10   # Should reach DMZ-Web through the VPN
```

Step 4.7 Verify the traffic is actually encrypted

On the host running pfSense, capture traffic on the WAN interface while the VPN is active and the client is browsing:

```
# From the pfSense console (Diagnostics → Packet Capture)
Interface: WAN
Host Address: <client public IP>
Protocol: UDP
Port: 51820
```

The captured packets should show only UDP/51820 traffic, with payloads that look like random bytes. Compare to a capture of unencrypted HTTP traffic from the same source — the contrast makes the value of encryption visible in a way that no amount of theory can match.

PART 5

DNS-Layer Protection with Pi-hole

Estimated time: **45 minutes**

Most malware and most phishing campaigns rely on DNS at some point — to reach a command-and-control server, to load malicious content, or to send a victim to a fake landing page. Blocking resolution of known-bad domains stops a meaningful fraction of attacks before they ever produce a packet. Pi-hole is a free DNS sinkhole that pulls block lists from public sources, runs as a recursive DNS server, and returns a non-routable address for any name that matches a list.

Step 5.1 Install Pi-hole on MGMT-Tools

On the MGMT-Tools VM, install Pi-hole using its automated installer:

```
curl -sSL https://install.pi-hole.net | bash
```

During the installer:

- Select the network interface (likely ens3) for Pi-hole to listen on.
- Choose an upstream DNS provider — Cloudflare (1.1.1.1) or Quad9 (9.9.9.9) are good defaults.
- Accept the default block lists.
- Install the web admin interface and lighttpd.
- Note the admin password printed at the end of installation; you will need it.

Step 5.2 Verify Pi-hole is resolving

From the MGMT-Tools VM:

```
dig @127.0.0.1 example.com
# Should return the IP for example.com
```

```
dig @127.0.0.1 doubleclick.net
# Should return 0.0.0.0 (or similar) — the domain is on a block list
```

Step 5.3 Point pfSense at Pi-hole

58. In the pfSense GUI, navigate to System → General Setup.
59. Set the DNS Servers to 192.168.40.10 (the Pi-hole address).
60. Uncheck Allow DNS server list to be overridden by DHCP/PPP on WAN.
61. Save.
62. Navigate to Services → DNS Resolver. Disable the resolver, or configure it to forward all queries to 192.168.40.10. Save and Apply.
63. Navigate to Services → DHCP Server → LAN. Set the DNS Server option to 192.168.40.10. Repeat for DMZ and MGMT. Save.

Step 5.4 Renew DHCP on clients and validate

On each client, release and renew the DHCP lease so the new DNS server takes effect:

```
# Linux
sudo dhclient -r ens3 && sudo dhclient ens3

# Confirm the resolver
nmcli dev show | grep DNS
# or
cat /etc/resolv.conf
```

Browse a few sites from LAN-Client. Then open the Pi-hole admin interface at http://192.168.40.10/admin and log in. You should see:

- A live count of queries from the LAN-Client IP.
- A breakdown of allowed vs. blocked queries.
- A top-domains list dominated by tracker and ad domains, most of them blocked.

Step 5.5 Add a custom block list and test

64. In the Pi-hole admin GUI, navigate to Adlists.
65. Add the URL https://hosts-file.net/grm.txt (or any other community block list — search for "Pi-hole block lists").
66. Click Tools → Update Gravity to refresh the lists.
67. From LAN-Client, attempt to resolve a domain that should now be blocked, and confirm the response.

PART 6

Traffic Capture and Analysis with Wireshark

Estimated time: **60 minutes**

Network security depends on visibility. Logs and alerts tell you what tools noticed; a packet capture tells you what actually happened on the wire. Wireshark is the de facto open-source tool for this, and a basic facility with capture filters and display filters separates the engineers who can answer hard questions from those who cannot.

Step 6.1 Install Wireshark on LAN-Client

```
sudo apt update
sudo apt install wireshark -y
# When prompted "Should non-superusers be able to capture packets?" answer Yes.
sudo usermod -aG wireshark $USER
# Log out and back in for the group change to take effect.
```

Step 6.2 Capture an HTTP session

68. Open Wireshark.
69. Double-click the network interface (likely ens3) to begin capturing.
70. In a terminal, run:

```
curl http://192.168.20.10
```

71. Stop the capture in Wireshark.
72. Apply the display filter: http
73. Click on the GET request. In the lower pane, expand the Hypertext Transfer Protocol section. Note that you can read the request line and headers in plaintext.
74. Right-click the packet → Follow → HTTP Stream. The full request and response, including the HTML body, are reconstructed into a single readable view. Anything you sent over plain HTTP — credentials, session cookies, form data — is visible to anyone with this capture.

Step 6.3 Capture an HTTPS session and observe the difference

75. Start a new capture.
76. In a terminal, run:

```
curl https://example.com
```

77. Stop the capture and apply the filter: tls

78. Examine the traffic. You can see the TLS handshake (ClientHello, ServerHello, certificate exchange), but the application data is encrypted — there are no readable headers or content.
79. Apply the filter: tls.handshake.type == 1 to show only ClientHello messages. The Server Name Indication (SNI) field is visible in plaintext, which is why DNS-over-HTTPS and Encrypted Client Hello matter as next-generation privacy controls.

Step 6.4 Useful filters for routine analysis

Filter	Purpose
ip.addr == 192.168.20.10	Show all traffic to or from a specific host
tcp.port == 80 or tcp.port == 443	Show web traffic
dns	Show only DNS requests and responses
icmp	Show ping and other ICMP messages
tcp.flags.syn == 1 and tcp.flags.ack == 0	SYN-only packets — useful for spotting port scans
http.request.method == "POST"	Find form submissions
frame contains "password"	Find packets with the literal string (illustrates HTTP exposure)

Table 7 – IDS/IPS Ruleset

Step 6.5 Capture a port scan and recognize it

On the Attacker VM, run a basic Nmap scan against the DMZ web server (this is benign — you control both ends):

```
sudo nmap -sS -p 1-1000 192.168.20.10
```

On LAN-Client, capture traffic during the scan (or capture from pfSense's Diagnostics → Packet Capture). Apply the filter:

```
tcp.flags.syn == 1 and tcp.flags.ack == 0 and ip.src == 192.168.50.10
```

You should see hundreds of SYN packets to sequential ports — the textbook signature of a SYN scan. This is also what an IDS rule against scanning is matching on.

PART 7

Centralized Logging

Estimated time: **45 minutes**

Every tool in this lab — pfSense, Suricata, Pi-hole, the endpoint VMs — writes logs locally. Local logs are useful until they are not, which is the moment something compromises a host and the first thing it does is delete the evidence. Centralized logging shifts that risk: the logs leave the host as they are written, and an attacker would need to also compromise the central collector to remove them.

Step 7.1 Configure rsyslog on MGMT-Tools as a collector

On MGMT-Tools, enable rsyslog to listen on UDP/514:

```
sudo tee /etc/rsyslog.d/10-remote.conf <<'EOF'
# Accept syslog from the lab network over UDP
module(load="imudp")
input(type="imudp" port="514")

# Write each remote source to its own file under /var/log/remote/
template(name="LabFormat" type="string"
  string="/var/log/remote/%fromhost-ip%/%programname%.log")
ruleset(name="lab") {
  action(type="omfile" dynaFile="LabFormat")
  stop
}
input(type="imudp" port="514" ruleset="lab")
EOF

sudo mkdir -p /var/log/remote
sudo systemctl restart rsyslog
```

Open the firewall on MGMT-Tools to allow UDP/514 from the lab subnets:

```
sudo ufw allow from 192.168.0.0/16 to any port 514 proto udp
sudo ufw status
```

Step 7.2 Configure pfSense to forward logs

80. In the pfSense GUI, navigate to Status → System Logs → Settings.
81. Scroll to Remote Logging Options.
82. Check Enable Remote Logging.
83. Source Address: LAN (or any interface that can reach 192.168.40.10).

84. Remote log servers: 192.168.40.10:514.
85. Remote Syslog Contents: select Everything (or at least Firewall Events, System Events, DHCP, and Suricata).
86. Save.

Step 7.3 Configure endpoints to forward logs

On each Ubuntu host, add a forwarding line to rsyslog:

```
echo "*.* @192.168.40.10:514" | sudo tee -a /etc/rsyslog.d/50-forward.conf
sudo systemctl restart rsyslog
```

Step 7.4 Verify log delivery

On MGMT-Tools, list the directory you configured for incoming logs:

```
ls -la /var/log/remote/
# Expected: a subdirectory per source IP, each containing one or more .log files
```

Generate a meaningful event from another host (for example, a failed sudo attempt on LAN-Client) and confirm the corresponding log line appears within seconds on MGMT-Tools:

```
sudo tail -f /var/log/remote/192.168.10.10/sudo.log
```

Step 7.5 Write a basic detection query

Centralized logging without queries is just an expensive disk. Write a simple grep-based detection that surfaces failed SSH logins from across the environment:

```
grep -r "Failed password" /var/log/remote/ | \
  awk '{print $1, $7, $11}' | \
  sort | uniq -c | sort -rn | head -20
```

Run an SSH brute-force simulation from the Attacker VM and watch the count rise:

```
# From Attacker VM (benign; rate-limited)
for i in $(seq 1 5); do
  sshpass -p "wrong" ssh -o StrictHostKeyChecking=no \
    -o PreferredAuthentications=password \
    user@192.168.10.10 'whoami' 2>/dev/null
done
```

This is the embryonic form of a SIEM detection rule. Substituting Splunk, Elastic, or Graylog for grep changes the scale, not the shape, of the work.

PART 8

End-to-End Validation Scenario

Estimated time: **60 minutes**

With every layer in place, walk through a complete attack scenario and observe how each control responds. The scenario is benign — you control both ends, and nothing in this lab is exposed beyond your host machine — but the activity it produces is realistic enough to exercise every tool you have configured.

The Scenario

An attacker on the lab's external network performs reconnaissance against the DMZ web server, then attempts to deliver a known exploit payload. Your goal is to identify which control caught which step.

Step 8.1 Prepare the Attacker VM

Place the Attacker VM on a fourth host-only network (LAB-ATTACK, 192.168.50.0/24) and add a temporary pfSense rule allowing 192.168.50.0/24 to reach the DMZ on HTTP/HTTPS. This simulates internet-side traffic without actually exposing anything.

Step 8.2 Reconnaissance

From the Attacker VM:

```
sudo nmap -sS -p- --min-rate=2000 192.168.20.10
```

Then check:

- Suricata alerts: a port scan signature should fire (ET SCAN or similar).
- pfSense firewall log on MGMT-Tools: many denied attempts to non-allowed ports.
- ntopng or simple bandwidth view: a sudden surge of small connections from 192.168.50.10.

Step 8.3 Banner grabbing and identification

```
curl -I http://192.168.20.10
nikto -h http://192.168.20.10
```

Nikto runs many requests against known web vulnerabilities and produces dozens of Suricata alerts. Open the alerts page in pfSense and observe the volume.

Step 8.4 Simulated payload delivery

```
# Triggers a known-bad pattern; benign content but matches an ET signature
```

```
curl -A "() { :;}; /bin/echo Shellshock" http://192.168.20.10/
```

This sends a Shellshock-style User-Agent. Suricata should fire an ET WEB_SERVER signature. The web server itself is patched (Ubuntu 22.04 is not vulnerable), so the attack does not succeed — but the detection demonstrates that the IDS catches the attempt regardless of outcome.

Step 8.5 Pivot attempt and DNS exfiltration

Simulate an attacker on the DMZ trying to reach the LAN, and a malicious lookup:

```
# From DMZ-Web (assume the attacker has compromised it)
ping -c 2 192.168.10.10          # Should be blocked by firewall
dig malware-test.example         # Should be blocked or sinkholed by Pi-hole
```

Both attempts produce telemetry: the firewall log shows the denied LAN ping, and the Pi-hole query log shows the suspicious lookup.

Step 8.6 Build the post-incident timeline

Using only the centralized logs and Suricata alerts, reconstruct the attack as a chronological timeline:

87. First reconnaissance: time, source IP, what was scanned.
88. First Suricata alert: time, signature, source IP.
89. First firewall denial: time, source/destination, port.
90. Web vulnerability scan: time, number of attempts.
91. Payload delivery attempt: signature triggered, response from server.
92. Pivot attempt: time, source, destination, blocking control.
93. DNS lookup attempt: time, resolved by/blocked by which control.

This timeline is the artifact a real incident responder produces in the first hour of an investigation. The fact that you can produce it from telemetry alone is the strongest possible validation that the controls in this lab are doing their job.

9. Troubleshooting Reference

Symptom	Likely Cause and Fix
A VM cannot reach pfSense	Confirm the VM is on the correct host-only network and that pfSense's interface for that zone is enabled and has an IP.
Suricata alerts are empty after generating test traffic	Suricata may still be loading rules. Wait 60 seconds and recheck. Confirm Suricata is running on WAN (green status icon). Confirm Send Alerts to System Log is checked.
WireGuard client cannot connect	Check the WAN firewall rule for UDP/51820. Confirm the keys on both sides match. Confirm the client's Endpoint matches the pfSense WAN address (not LAN).
Pi-hole admin shows zero queries	Clients may still be using the previous DNS server. Force a renewal (sudo dhclient -r && sudo dhclient) and check /etc/resolv.conf points at 192.168.40.10.
Logs are not appearing on MGMT-Tools	Confirm rsyslog is listening (sudo ss -lup | grep 514). Confirm the firewall on MGMT-Tools allows UDP/514. Confirm pfSense's source address can reach 192.168.40.10.
Wireshark shows no packets	Confirm you selected the right interface and that you are not capturing on the loopback interface by mistake. Some laptops require the user to be in the wireshark group; log out and back in after adding.
Nmap scan from Attacker shows all ports filtered	The pfSense firewall is dropping the SYNs before they reach the host. This is the correct behavior — confirm by checking the firewall log for matching denials.

Table 8 – Troubleshooting FAQ

10. Lab Deliverables

Produce the following artifacts to consider the lab complete:

94. A topology diagram annotated with the four zones, their subnets, and the location of each control (firewall, Suricata, Pi-hole, log collector, VPN endpoint).
95. Screenshots of the Suricata alerts page showing at least three distinct signatures triggered during the validation scenario.
96. A copy of the WireGuard client configuration file (with private keys redacted) and a packet capture excerpt confirming traffic is encrypted on the wire.
97. A screenshot of the Pi-hole dashboard showing query volume, block rate, and at least one custom block list.

98. A screenshot of the LAN-Client Wireshark capture comparing an HTTP request (plaintext visible) and an HTTPS request (encrypted).
99. The output of the failed-SSH detection query from Step 7.5, run after the validation scenario.
100. A written timeline (about one page) reconstructing the attack from Part 8 using only telemetry, with each step tagged by which control caught it.

11. Extending the Lab

Each of the following extensions deepens a specific layer:

- **TLS interception with Squid.** Configure Squid as a transparent proxy on pfSense and bump TLS for outbound web traffic. Discover for yourself why this is operationally hard and ethically charged.
- **Network flow analytics with ntopng.** Install the ntopng package and explore flow data. Compare what flow visibility tells you that packet capture does not.
- **Honeypot in the DMZ.** Deploy a small honeypot (Cowrie for SSH, or a simple TCP listener) in the DMZ and watch what hits it during the validation scenario.
- **ELK stack instead of rsyslog.** Replace the simple file-based collector in Part 7 with an Elasticsearch + Logstash + Kibana stack and write Kibana queries instead of grep.
- **Zero-trust segmentation with WireGuard mesh.** Replace the broad VPN allow rule with peer-to-peer WireGuard tunnels between specific hosts, and require authentication at every hop.
- **Automated alerting.** Wire Suricata alerts into a chat webhook (Slack, Discord, Mattermost) so that high-severity signatures produce a real-time notification.

12. Conclusion

A working defense-in-depth network is the sum of many ordinary configurations. Each control in this lab was modest on its own — a firewall rule, an IDS signature, a DNS block list, a syslog forward — but together they produced an environment where an attacker leaves visible footprints at every step, and where the failure of any one control does not collapse the whole posture.

The configurations themselves are less valuable than the habit of building them. Returning to this lab whenever a new tool, technique, or threat needs evaluation gives you a place to test before production sees the change. The specific products will evolve — pfSense may become OPNsense, Suricata may give way to Zeek-led detection, Pi-hole may yield to commercial DNS firewalls — but the architecture of layered, observable, auditable controls is durable. That architecture is what this lab teaches; the rest is implementation detail.